

Tharun SV

Paper

 papers
 IV CYS
 Amrita Vishwa Vidyapeetham

Document Details

Submission ID**trn:oid::1:3043554607****Submission Date****Oct 16, 2024, 11:08 AM GMT+5:30****Download Date****Oct 16, 2024, 11:10 AM GMT+5:30****File Name****conference-paper.docx****File Size****811.0 KB****6 Pages****4,535 Words****27,858 Characters**

71% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups

- 
1 AI-generated only 71%
 Likely AI-generated text from a large-language model.
- 
2 AI-generated text that was AI-paraphrased 0%
 Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely AI generated as AI generated and AI paraphrased or likely AI generated and AI paraphrased writing as only AI generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



Zero Trust Security for Web Applications in Microservice-Based Environments

Jayaraj Viswanathan

Department of Computer Science and Engineering,
Amrita Vishwa Vidyapeetham,
Chennai, India,
ch.en.u4cys21026@ch.students.amrita.edu

Dinesh Kumar N

Department of Computer Science and Engineering,
Amrita Vishwa Vidyapeetham,
Chennai, India,
ch.en.u4cys21014@ch.students.amrita.edu

Abstract—In the rapidly evolving landscape of cybersecurity, the increasing complexity and frequency of cyber threats have rendered traditional perimeter-based security models insufficient. As enterprises shift towards cloud-native applications and microservice architectures, a more robust security framework is essential. This paper introduces a Zero Trust Security Model for web applications in Kubernetes clusters, utilizing key technologies such as JSON Web Token (JWT) authentication and mutual Transport Layer Security (mTLS). Unlike traditional models that rely on predefined trust zones, this approach enforces continuous identity verification, dynamic access control, and fine-grained micro-segmentation to secure sensitive data. The model incorporates Zero Trust principles, including encrypted communications, strict user and device authentication, and real-time threat detection. It provides robust policy enforcement to protect APIs, containers, databases, and other critical resources in a microservices environment. Continuous monitoring of user behavior, combined with machine learning algorithms for threat detection, enhances the system's ability to detect and mitigate potential incidents in real-time. Our experimental validation shows that this model effectively minimizes lateral movement and unauthorized access, significantly reducing attack surfaces. The proposed Zero Trust architecture offers a scalable, secure, and resilient framework suited to modern containerized applications, ensuring data integrity and security.

Keywords— Zero Trust Security, JWT Authentication, Mutual TLS, Cybersecurity, Web Application Security, Adaptive Access Control, Micro-segmentation, Data Protection, Cloud-Based Architecture, Kubernetes Clusters.

I. INTRODUCTION

In today's digital landscape, organizations have faced an escalating number of cyber threats, which have highlighted the inadequacies of traditional perimeter-based security measures. These models, often termed "castle-and-moat" defenses, operate under the assumption that threats primarily originate outside the network, rendering internal resources vulnerable. This oversimplification has led to significant breaches, such as the 2017 Equifax incident, which exposed personal information of approximately 147 million individuals and resulted in an estimated \$4 billion in remediation costs and legal settlements. Similarly, the 2020 SolarWinds cyberattack compromised multiple U.S. government agencies, further underscoring the urgent need for more robust security frameworks.

The rapid adoption of cloud computing, agile methodologies, and containerization has significantly expanded the attack surface, making organizations more

susceptible to sophisticated cyber threats. Attack vectors such as phishing, ransomware, and insider threats have surged, with ransomware attacks alone increasing by 150% in 2020, inflicting an estimated \$20 billion in damages. Furthermore, the Verizon 2020 Data Breach Investigations Report indicated that 86% of breaches were financially motivated, highlighting the critical need for adaptive and proactive security measures that can evolve alongside emerging threats. To counter these challenges, the Zero Trust Security Model has emerged as a transformative approach in cybersecurity. Unlike traditional models that rely on predefined trust zones and network location to establish trust, Zero Trust mandates that no entity—be it user, device, or application—be trusted by default. Every access request must undergo continuous authentication and authorization, regardless of its origin. This "never trust, always verify" philosophy is particularly relevant in microservice-based environments, where data and services are distributed across multiple containers and accessed by users from varied locations. This paper explores the design and implementation of a Zero Trust framework within a modern web application utilizing a microservice architecture. By employing key technologies such as JSON Web Tokens (JWT) for stateless authentication, mutual TLS (mTLS) for secure communication, and micro-segmentation to isolate services, the proposed framework enhances security by continuously verifying all entities and enforcing strict access controls.

This significantly mitigates risks associated with lateral movement attacks, where an attacker gaining access to one part of the system can navigate freely across other containers, potentially leading to widespread data breaches. The motivation for adopting Zero Trust in Kubernetes clusters arises from the complexities of securing modern distributed systems. Traditional models often grant access based on network location, which is inadequate in environments where applications are highly decoupled and operate across various cloud providers. In these settings, vulnerabilities can be exploited, necessitating a model that safeguards distributed resources and sensitive data effectively. By addressing the limitations of traditional security models and adapting to the dynamic nature of cloud-native applications, this research contributes significantly to the field of cybersecurity. The proposed framework offers a scalable and adaptive solution to protect modern, containerized environments, ensuring the integrity and confidentiality of sensitive data across an organization's application ecosystem.

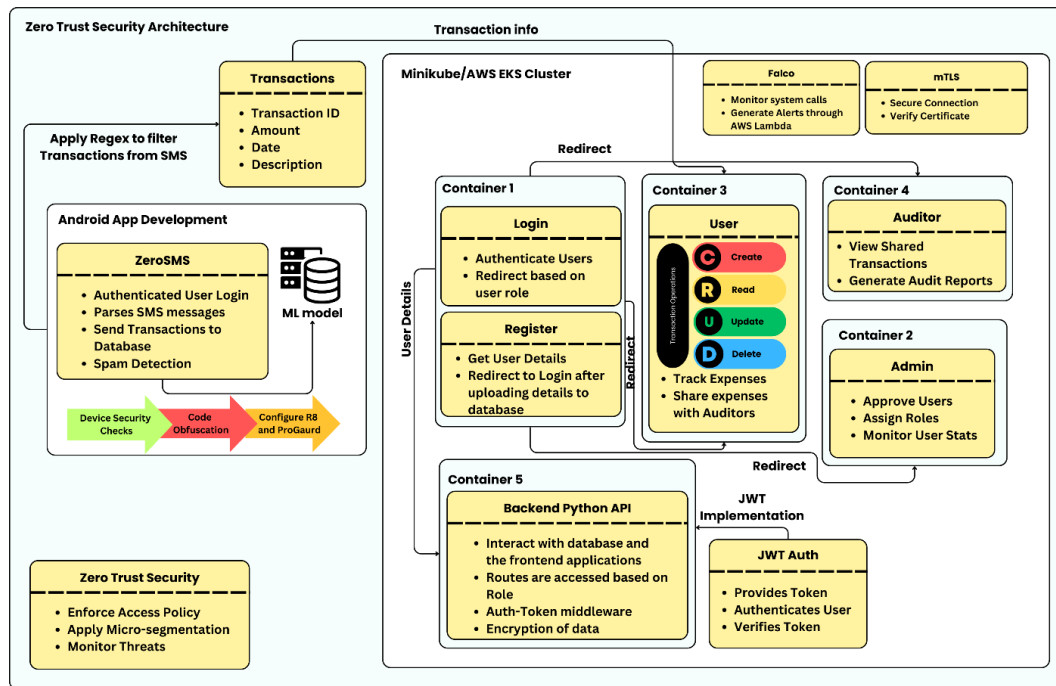


Fig. 1. Architecture diagram of the implementation of Zero Trust in the Web Application

II. RELATED WORKS

D'Silva et al. examined the implementation of Zero Trust Architecture (ZTA) using Kubernetes [7], highlighting its significance in cloud-based environments. They emphasized how ZTA continuously verified users, devices, and applications, challenging traditional trust models. The study discussed the use of technologies such as Kubernetes, Docker and RBAC/ABAC for enhanced access control. Additionally, they critiqued traditional security frameworks, noting the inadequacies of perimeter-based models in modern cloud infrastructures. Their proposed architecture improved security through continuous verification and logging, making it adaptable to decentralized systems.

Varalakshmi et al. [8] explored the enhancement of JSON Web Token (JWT) authentication within Software Defined Networks (SDNs). They identified vulnerabilities in traditional JWT implementations and proposed a modified authentication approach that improves security without sacrificing performance. Their analysis highlighted the importance of securing API endpoints and user sessions, ultimately enhancing the overall integrity of network communications in SDN environments. Bucko et al. [9] examined the enhancement of JWT authentication and authorization in web applications by utilizing user behavior history. Their study analyzed how behavior-based metrics could improve the reliability of authentication processes. They proposed a dynamic authentication mechanism that adapts based on user activity, significantly reducing unauthorized access and enhancing user security in web applications.

Jánoky et al. [10] conducted an analysis of revocation mechanisms for JSON Web Tokens (JWTs). They identified the challenges associated with effectively revoking tokens without degrading system performance. Their research emphasized the need for efficient revocation strategies to ensure the security of applications relying on

JWTs, contributing valuable insights into the management of token lifecycles. Achary and Shelke [11] investigated fraud detection in banking transactions using machine learning techniques. They presented a framework that leveraged various algorithms to identify and mitigate fraudulent activities in real-time. Their findings underscored the efficacy of machine learning in enhancing the security of financial transactions, providing banks with tools to protect against fraud. The authors highlighted the importance of adaptive models that can learn from evolving fraud patterns, contributing to more robust credit card security measures.

Fraudulent Transaction Detection in Credit Card by Applying Ensemble Machine Learning Techniques by Prusti et al. [12] explores the use of ensemble machine learning models for detecting fraudulent credit card transactions. By combining multiple machine learning algorithms, the study aims to improve accuracy, precision, and detection speed. Key findings show that ensemble methods outperform individual classifiers in identifying fraudulent transactions, reducing false positives, and increasing overall efficiency in real-time fraud detection systems. Jaculine Priya and Saradha [13] reviewed machine learning algorithms for fraud detection and prevention. Their comprehensive analysis encompassed various techniques, assessing their strengths and weaknesses in different contexts. The paper provided a critical overview of the current state of fraud detection systems, emphasizing the potential of machine learning in enhancing predictive accuracy and operational efficiency in fraud management.

A. I. Weinberg et al. [14] studied Zero Trust Architecture (ZTA) for applications and network security, highlighting its relevance in contemporary cybersecurity frameworks. The research detailed how ZTA eliminates implicit trust and enforces strict access controls, providing a robust model for securing sensitive data in dynamic environments. Kang et al. [15] provided a brief survey of the theory and application of Zero Trust security. They examined various

implementations and discussed how Zero Trust principles could enhance overall security posture. Their findings illustrated the applicability of Zero Trust in diverse settings, emphasizing its importance in mitigating modern cybersecurity threats.

Ashfaq et al. [16] presented a comprehensive survey of the Zero Trust security paradigm. Their research analyzed existing literature, outlining key principles and strategies for implementing Zero Trust in various contexts. The authors identified gaps in current research, suggesting areas for future investigation to enhance understanding and application of Zero Trust principles. Liu et al. [17] dissected the research landscape surrounding Zero Trust and its implementation in the Internet of Things (IoT). They identified key challenges and strategies for applying Zero Trust principles in IoT environments, emphasizing the necessity for stringent access controls and continuous verification to mitigate security risks. The authors provided a comprehensive overview of current research trends and highlighted areas requiring further investigation to enhance IoT security through Zero Trust frameworks.

Mehraj and Banday [18] proposed a Zero Trust strategy for cloud computing environments to address security challenges such as identity theft, data breaches, and trust management issues. Their model emphasizes strong access control and continuous verification due to the dynamic, shared nature of cloud services. The authors discussed how traditional security approaches are insufficient in cloud deployments and highlighted the Importance of Zero Trust in tracking and mitigating both external and insider threats. Muddinagiri et al. [19] presented a method for deploying Kubernetes locally using Minikube to manage Docker containers, emphasizing the advantages of containerization as a lightweight alternative to virtual machines. The paper highlights the importance of local Kubernetes testing for industries like finance and healthcare, which require secure, scalable applications without relying on cloud-based deployments. Their approach was demonstrated using a Python-based web server built with a DockerFile. Pace's thesis [20] focuses on the implementation of Zero Trust Networks using Istio in Kubernetes environments. Istio, as a service mesh, manages traffic, observability, and security, offering features like mutual TLS, JWT-based authentication, and seamless integration with Kubernetes, enhancing microservices security and operational efficiency.

III. Methodology

The proposed containerized web application is designed to provide users with a comprehensive expense tracking system that adheres to Zero Trust security principles as illustrated in Figure 2. This application uses Docker for containerization, Kubernetes for Container Orchestration and Helm to dynamically handle configurations of Kubernetes manifests. The complete application is then deployed to AWS EKS cluster remotely enforced with strict IAM policies and VPC.

A. Web Application Overview

At the core of this application is the user interface, which enables users to perform CRUD operations (Create, Read, Update, Delete) on their financial transactions. Users can manually input their expenses, savings, and investments through a form-based system that adds each entry as a card component. A distinguishing feature of this application is

its integration with the ZeroSMS mobile app. The ZeroSMS app automatically parses SMS messages related to transactions, which are often sent by banks or payment services, and synchronizes them with the web application. This functionality is especially beneficial for users who may forget to log some expenses or are too busy to manually enter transaction details. Another key feature is to the ability for users to share their transaction data with a selected list of auditors. The auditors, who are granted read-only access to user transactions, can view the data to prepare financial reports. The auditor's interface displays

list of users who have shared their transaction data, allowing them to access the data without modifying it, which aligns with Zero Trust's principle of minimal privileges for authorized users.

A. Admin and Auditor Interfaces

The admin interface is critical for managing user roles and access within the application. New users must first register through a registration page, but their access to the system is only granted upon approval by the admin. The admin has the authority to approve or reject users, as well as assign specific roles such as user, auditor, or admin. Role-based access control (RBAC) ensures that each user only has access to the resources necessary for their role, supporting the Zero Trust model's principle of least privilege. In addition to user management, the admin interface provides a user behavior analysis dashboard. This feature enables the admin to monitor important logs, such as user login and logout times, system activity, and other behaviors that may indicate suspicious activity. By continuously monitoring these behaviors, the application can detect anomalies, helping to prevent unauthorized access or potential misuse of the system as shown in Figure 4. This feature reinforces Zero Trust's "never trust, always verify" approach, where users are continuously authenticated and their behavior is scrutinized to detect potential threats.

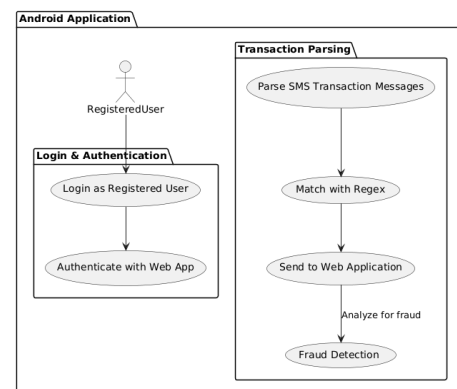


Fig. 2. Activity Diagram for the ZeroSMS Android Application

B. Access Tokens functionality and Implementation

Each request to access sensitive data is validated JSON Web Tokens (JWT), which provide secure, time-bound access tokens that validate the user's identity and role. If a user's JWT token is invalid or expired, access is immediately denied, ensuring that no unauthorized requests are processed. JWTs are compact, URL-safe tokens that enable secure information exchange between parties. Comprising three parts—the header, which defines the signing algorithm; the payload, which contains user claims; and the signature, used to verify the token's authenticity. This structure allows for seamless authentication flows, where the server can validate the token without maintaining session information.

The Python APIs to generate and manage and rigorously validate JWT are containerized and built to run as a separate pod inside the Kubernetes cluster which ensures continuous authentication of requests from the clients. Upon successful authentication, the system generates a JWT containing essential user-specific claims, such as roles and permissions, which is then transmitted to the user. For subsequent requests, the user includes this token in the authorization header, allowing the application to verify the token's integrity and validity before granting access to protected resources. This implementation not only adheres to the principle of least privilege by limiting access based on the user's current permissions but also enhances security through the use of short-lived access tokens which mitigate the risks associated with token theft or misuse.

C. Spam Detection in Financial Transactions

The ZeroSMS app parses the SMS messages and uses regex to classify them as either transaction-related or non-transaction-related. This functionality not only helps users track their expenses more effectively but also mitigates the risk of overlooking minor transactions such as small purchases and pocket money. As illustrated in Figure 8, the workflow demonstrates how SMS parsing feeds into the user's expense management process, emphasizing the seamless integration between the mobile and web platforms. To further enhance the functionality of the ZeroSMS app, a Machine Learning (ML) model was developed to detect spam messages. The model was trained on spam SMS dataset from Kaggle and built using a combination of supervised learning techniques and ensemble methods, specifically leveraging algorithms such as Random Forest, Gradient Boosting, and Support Vector Machines (SVM). After extensive testing, the model achieved an accuracy of over 96%, demonstrating its effectiveness in identifying potential fraud. As shown Table 1, the performance metrics highlight the model's precision and recall rates, emphasizing its capability to minimize false positives and negatives in spam detection.

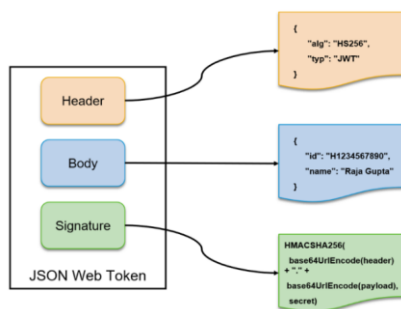


Fig. 3. JWT Architecture

D. Policy Enforcement Point for RBAC

A Policy Enforcement Point (PEP) is a crucial component in enforcing security within web applications, especially in the context of a Zero Trust Architecture (ZTA). The PEP acts as a gatekeeper that intercepts all incoming requests before they reach the application's core services, ensuring that only authorized users or services gain access. When implemented in microservice based web applications, the PEP evaluates each request based on predefined security policies. These policies determine who can access which service and resources, at what time, and under what conditions. The PEP typically works alongside a Policy Decision Point (PDP), which makes the decision regarding access based on rules such as role-based access control (RBAC) or attribute-based access control (ABAC) as

shown in Figure 3. Once a decision is made by the PDP, the PEP either grants or denies access.

In practical terms, the PEP is embedded at critical points within the microservice architecture, such as API gateways or middleware layers. Every incoming request to the service of application containers passes through the PEP, where it is authenticated, and authorization policies are applied. If the user or service is unauthorized or violates policies, the PEP blocks access. For an instance, in our application a, the PEP ensures that an auditor is allowed to access only the transactions and data related to their client. Also, the routes behind the services are restricted and are specific to the entities (user, admin, auditor) to access. It could also enforce least privilege access, allowing users or services to only perform the actions necessary for their role. Additionally, PEPs can be deployed as sidecar containers in microservices, ensuring that access controls are enforced at each service endpoint. This is especially important when services communicate internally within distributed environments. Overall, PEP strengthens security by ensuring consistent policy enforcement, providing granular access control, and mitigating potential attack vectors.

E. mTLS within the Kubernetes Clusters

Mutual TLS (mTLS) is a foundational security mechanism in implementing Zero Trust Architecture (ZTA) for microservices [21]. It provides robust authentication and encrypted communication between microservices by requiring both the client and server to present valid certificates, ensuring mutual trust before communication occurs. This mutual verification process is key to the "never trust, always verify" principle of ZTA. In mTLS, each pod in the cluster is assigned a unique cryptographic certificate issued by a trusted Certificate Authority (CA). When two services communicate, both sides (client and server) present their certificates during the handshake process. Each service verifies the other's certificate against the CA's public key to ensure authenticity. Once both certificates are validated, an encrypted communication channel is established using TLS (Transport Layer Security).

This ensures that data exchanged between services is not only authenticated but also protected from eavesdropping and tampering. mTLS is integrated into our application service meshes using a tool named *Istio* which automate certificate management and enforce mTLS at the network layer. In these environments, mTLS is applied transparently to microservices, without requiring each service to implement its own TLS logic. The service mesh handles certificate issuance, renewal, and rotation, ensuring seamless mTLS enforcement. Unlike traditional systems that assume trust once inside the network, mTLS requires that every interaction between services involves identity verification [22]. This eliminates the risk of unauthorized services accessing resources, as only those with valid certificates are allowed to communicate. When combined with Policy Enforcement Points (PEPs), mTLS strengthens the overall ZTA. The PEP verifies that requests originate from authorized services with valid certificates before granting access to resources.

IV. RESULTS

The implementation of the Zero Trust application successfully demonstrated the core principles of Zero Trust Architecture (ZTA) by ensuring strict access control, continuous authentication, and verification of users and devices. The system effectively secured sensitive resources

by integrating role-based access control (RBAC) and least-privilege policies. The integration of Kubernetes for container orchestration further enhanced scalability and resource management, allowing the system to adapt to the changing security requirements in real-time. Overall, the implementation exhibited robust security postures with minimal performance overhead. The web application was deployed in an EKS cluster, utilizing node groups belonging to the EKS Cluster. As illustrated in Figure 4., the architecture ensures scalability and fault tolerance, with seamless communication between services for a smooth user experience.

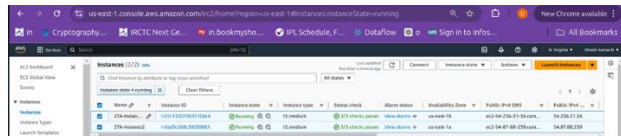


Fig 4. Node Groups of EKS Cluster deployed as EC2 instances

Security within the application is strengthened through AWS Lambda and the Falco tool, which monitors the cluster for anomalies and triggers alerts for real-time threat detection, as illustrated in Figure 5. showcases the alerts from Lambda as logs captured in AWS Cloudwatch. These logs contain valuable information such as the violated rule, container name, container ID, pod name, and namespace where the attack occurred, along with the system call used to perform the attack. This collective data emphasizes the implementation of the Zero Trust strategy and aids in identifying the root cause of suspicious activity. Additionally, session logs provide a thorough audit trail, tracking user activities and identifying suspicious behavior. These logs ensure compliance with Zero Trust principles by ensuring all user actions are continuously monitored and recorded.

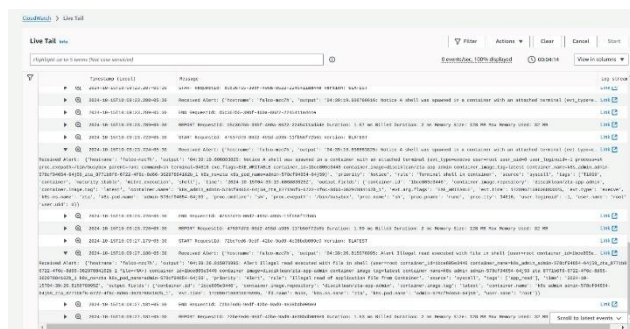


Fig 5. AWS Lambda Alerts triggered by Falco tool deployed in the cluster

As far as the Spam detection model is considered, Naive Bayes was the best model with an F1 score of 0.918 and accuracy of 0.978 as shown in Table 1, showing a strong balance of precision and recall. SVM had the highest precision (1.0) and ROC AUC score (0.984), while Logistic Regression also performed well with an F1 score of 0.909. Random Forest and Gradient Boosting had slightly lower F1 scores but maintained high accuracy across all models.

To prepare for production release, the app leverages the JailMonkey library to detect rooting and unsafe environments, while utilizing R8 and ProGuard tools for code optimization and obfuscation to enhance performance and security as shown in Figure 10. The APK is securely signed with a private key from a trusted keystore, ensuring authorized distribution. Due to R8 and ProGuard obfuscation, reverse engineering ZeroSMS is significantly more challenging as illustrated in Figure 11.

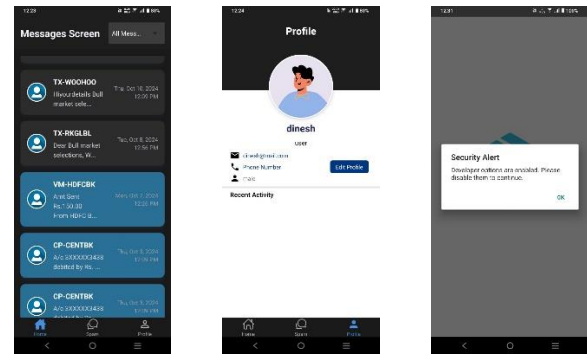


Fig. 6. ZeroSMS in production release with security features

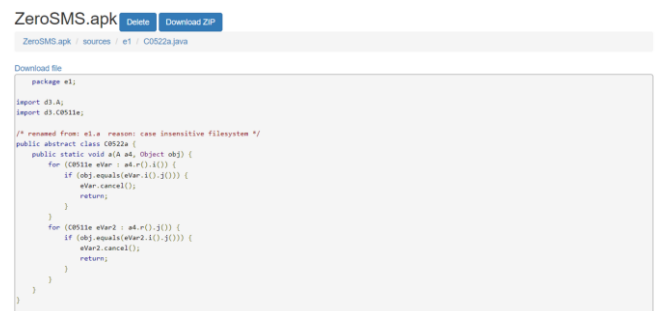


Fig. 7. ZeroSMS with class name and code Obfuscation

The MobSF scan illustrated in Figures 13 a) & b) shows ZeroSMS achieving a security score of 57 (Grade B), 83.87% higher than a standard APK, which scored 31 (Grade C). ZeroSMS has SMS permissions and allows "http" traffic for backend connectivity, impacting its security score. In contrast, the standard APK is merely a "Hello World" application built in Android Studio, lacking any additional privileges.

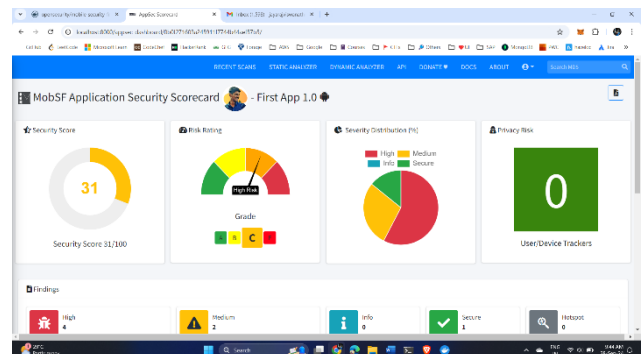


Fig. 8. a) MobSF scan result of standard APK

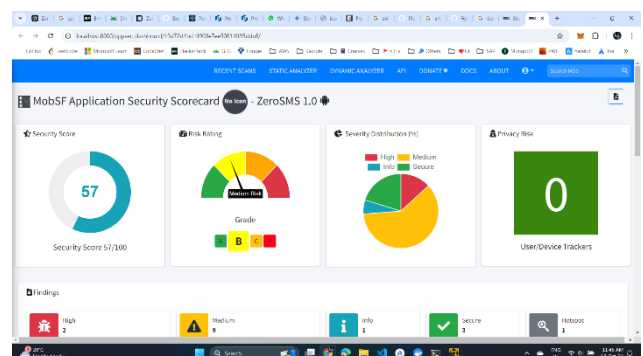


Fig. 8. b) MobSF scan result of ZeroSMS

Table 1. Comparison of Model Performance Metrics

Model	F1 Score	Recall	Precision	Accuracy	Mean Squared Error	ROC AUC Score	Mean Absolute Error	Specificity
Naive Bayes	0.918	0.918	0.918	0.978	0.022	0.975	0.022	0.988
SVM	0.9505	0.826	1.0	0.977	0.023	0.984	0.023	1.0
Logistic Regression	0.909	0.845	0.984	0.978	0.022	0.983	0.022	0.998
Random Forest	0.905	0.826	1.0	0.977	0.023	0.981	0.023	1.0
Gradient Boosting	0.87	0.781	0.983	0.969	0.031	0.973	0.031	0.998

V. CONCLUSION AND FUTURE WORK

The implementation of Zero Trust Architecture (ZTA) marks a significant shift in cybersecurity strategies, particularly for cloud-based and decentralized systems. Traditional perimeter-based security models have proven inadequate against sophisticated cyber threats and the challenges posed by remote work. By eliminating implicit trust and enforcing continuous verification of users and devices, ZTA addresses vulnerabilities in legacy frameworks. Technologies like Kubernetes and JSON Web Tokens (JWT) are vital in deploying Zero Trust principles, enabling fine-grained access control and dynamic policy enforcement. Continuous logging and monitoring enhance an organization's ability to promptly detect and respond to threats, fostering a security-aware culture. As organizations embrace digital transformation, adopting Zero Trust models will be crucial in mitigating risks associated with modern computing environments. Future research should focus on the challenges of implementing ZTA across various contexts, such as the Internet of Things (IoT) and decentralized identity management. Enhancements to the Zero Trust application could include advanced behavioral analytics for detecting anomalous activities and integrating machine learning algorithms for adaptive risk scoring to adjust access privileges in real-time.

VI. REFERENCES

- [1] O. E. Imokhai, T. E. Emmanuel, A. A. Joshua, E. S. Emakhu, and S. Adebisi, "Zero Trust Architecture: Trend and Impact on Information Security," *International Journal of Emerging Technology and Advanced Engineering*, vol. 12, no. 7, pp. 87-92, July 2022.
- [2] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, *NIST Special Publication 800-207 Zero Trust Architecture*. Gaithersburg, MD: National Institute of Standards and Technology (NIST), 2020.
- [3] Y. He, D. Huang, L. Chen, Y. Ni, and X. Ma, "A Survey on Zero Trust Architecture: Challenges and Future Trends," *Wireless Communications and Mobile Computing*, vol. 2022, Article ID 6476274, 13 pages, 2022. DOI: <https://doi.org/10.1155/2022/6476274>.
- [4] A. Mustyala and S. Tatineni, "Advanced Security Mechanisms in Kubernetes: Isolation and Access Control Strategies," *ESP Journal of Engineering & Technology Advancements*, vol. 1, no. 2, pp. 45-52, 2021.
- [5] B. Pranoto, "Threat Mitigation in Containerized Environments," *Applied Research in Artificial Intelligence and Cloud Computing*, vol. 6, no. 8, pp. 142-151, 2023.
- [6] S. Bagheri, H. Kermabon-Bobinnec, S. Majumdar, Y. Jarraya, L. Wang, and M. Pourzandi, "Warping the Defence Timeline: Non-disruptive Proactive Attack Mitigation for Kubernetes Clusters," in *Proc. IEEE International Conference on Communications (ICC 2023)*, Rome, Italy, 28 May - 01 June, 2023, pp. 567-574.
- [7] D. D'Silva and D. D. Ambawade, "Building a Zero Trust Architecture Using Kubernetes," in *Proc. 2021 6th International Conference for Convergence in Technology (I2CT)*, Pune, India, Apr. 2021, pp. 1-5.
- [8] P. Varalakshmi, B. Guhan, P. V. Siva, T. Dhanush, and K. Saktheeswaran, "Improvising JSON Web Token Authentication in SDN," in *Proc. 2022 International Conference on Communication, Computing and Internet of Things (IC3IoT)*, Mar. 2022. DOI: [10.1109/IC3IoT53935.2022.9767873](https://doi.org/10.1109/IC3IoT53935.2022.9767873).
- [9] A. Bucko, K. Vishi, B. Krasniqi, and B. Rexha, "Enhancing JWT Authentication and Authorization in Web Applications Based on User Behavior History," *Computers*, vol. 12, no. 4, pp. 78-85, 2023. DOI: <https://doi.org/10.3390/computers12040078>.
- [10] L. V. Jánoky, J. Levendovszky, and P. Ekler, "An Analysis on the Revoking Mechanisms for JSON Web Tokens," *International Journal of Distributed Sensor Networks*, vol. 14, no. 9, Aug. 2018. DOI: [10.1177/1550147718801535](https://doi.org/10.1177/1550147718801535).
- [11] R. Achary and C. J. Shelke, "Fraud Detection in Banking Transactions Using Machine Learning," in *Proc. 2023 International Conference on Intelligent and Innovative Technologies in Computing, Electrical and Electronics (IITCEE)*, Jan. 2023.
- [12] D. Prusti and S. K. Rath, "Fraudulent Transaction Detection in Credit Card by Applying Ensemble Machine Learning Techniques," in *Proc. Jun 2019 10th International Conference on Computing, Communication and Networking Technologies (ICCCNT)*.
- [13] G. J. Priya and S. Saradha, "Fraud Detection and Prevention Using Machine Learning Algorithms: A Review," in *Proc. 2021 7th International Conference on Electrical Energy Systems (ICEES)*, Feb. 2021. DOI: [10.1109/ICEES51510.2021.9383631](https://doi.org/10.1109/ICEES51510.2021.9383631).
- [14] A. I. Weinberg and K. Cohen, "Zero Trust Implementation in the Emerging Technologies Era: A Survey," *Complex Engineering Systems*, vol. 4, pp. 16-28, 2024. DOI: <http://dx.doi.org/10.20517/ces.2024.41>.
- [15] H. Kang, G. Liu, Q. Wang, L. Meng, and J. Liu, "Theory and Application of Zero Trust Security: A Brief Survey," *Entropy*, vol. 25, no. 12, 2023.
- [16] S. Ashfaq, S. A. Patil, S. Borde, P. Chandre, and P. M. Shafi, "Zero Trust Security Paradigm: A Comprehensive Survey and Research Analysis," *Journal of Electrical Systems*, vol. 19, no. 2, pp. 28-37, 2023.
- [17] C. Liu, R. Tan, Y. Wu, Y. Feng, Z. Jin, F. Zhang, Y. Liu, and Q. Liu, "Dissecting Zero Trust: Research Landscape and Its Implementation in IoT," *Cybersecurity*, vol. 7, no. 20, pp. 1-10, 2024. DOI: <https://doi.org/10.1186/s42400-024-00212-0>.
- [18] S. Mehraj and M. T. Banday, "Establishing a Zero Trust Strategy in Cloud Computing Environment," in *Proc. 2020 International Conference on Computer Communication and Informatics (ICCCI 2020)*, Coimbatore, India, Jan. 2020.
- [19] R. Muddanagiri, S. Ambavane, and S. Bayas, "Self-Hosted Kubernetes: Deploying Docker Containers Locally with Minikube," in *Proc. 2019 International Conference on Innovative Trends and Advances in Engineering and Technology (ICITAET)*.
- [20] M. Pace, *Zero Trust Networks with Istio*, Master's thesis, Politecnico di Torino, Turin, Italy, 2020.
- [21] A. Kurbatov, *Design and Implementation of Secure Communication Between Microservices*, Master's thesis, Aalto University, Espoo, Finland, 2020.
- [22] Y. Yang, W. Shen, B. Ruan, W. Liu, and K. Ren, "Security Challenges in the Container Cloud," in *Proc. 2021 Third IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications (TPS-ISA)*, Dec. 2021. DOI: [10.1109/TPS-ISA52974.2021](https://doi.org/10.1109/TPS-ISA52974.2021).